Close ×

Instructions

Overview

Assignment Overview:

This assignment forms an essential component of your final grades for the course. The assignment will be evaluated and graded by your instructor.

This assignment is graded out of a total of 10 points and will constitute 10% of your final module grade. The scoring for each question is provided at the beginning of the question.

The scope of this assignment spans material covered in pre-recorded lectures, group discussions, and live sessions.

Total time estimated for assignment completion: 24 hours

Assignment problem:

Problem Overview:

You are tasked with implementing a distributed banking system using gRPC and Protocol Buffers. The system needs to support multiple operations, such as retrieving account balances, transferring money between accounts, and viewing transaction history. These operations should be accessible through remote procedure calls (RPCs), and your implementation should be able to handle communication between distributed services.

You will implement a basic version of this system with two key services:

1. Account Management Service - Responsible for retrieving and updating account balances.
2. Transaction Service - Handles money transfers and transaction history.

Task:

1. **Design Protobuf Services:**
    - Define the gRPC services and their respective methods for account management and transactions.
    - The Account Management Service should have two methods:
        - getBalance: Retrieve the current balance of a user's account.
        - updateBalance: Update the balance after a transaction.
    - The Transaction Service should have two methods:
        - initiateTransfer: Initiates a transfer from one account to another.
        - getTransactionHistory: Retrieves a list of transactions for a user.
2. **Implement gRPC Server and Client:**
    - Implement a gRPC server in Python that handles the logic for the services.
    - Implement a gRPC client that can interact with the services and perform operations like checking balances, transferring money, and fetching transaction history.
3. **Define Protobuf Messages:**
    - Define appropriate Protobuf messages for each service method. Each message should contain necessary fields like user ID, amount, transaction details, etc.

4. **Test the RPC Communication:**
    - Use the gRPC client to interact with the services and test the system. Ensure that you can:
        - Retrieve and update balances.
        - Perform a money transfer between accounts.
        - Retrieve transaction history.
5. **Error Handling:**
    - Implement basic error handling for common issues like insufficient funds for transfer, non-existing users, and invalid data.

Deliverables

1. Submit the following at the appropriate prompts:
2. Your **Protobuf** definition file (banking.proto) and video recording of code explanation with your own voice-over
3. The **server-side** implementation code and video recording of code explanation with your own voice-over
4. The **client-side** implementation code and video recording of code explanation with your own voice-over
5. Video recording of demonstration of your application with your own voice-over.

Please note that this is not a GROUP exercise. You are required to work on this assignment on your own. Do not share code or do a verbatim copy of other's code.

No. of attempts: 1

The assignment will be graded with the following distribution of points:

1. Question 1 (2 out of 10): 20% of total points
2. Question 2 (3 out of 10): 30% of total points
3. Question 3 (3 out of 10): 30% of total points
4. Question 4 (2 out of 10): 20% of total points

Your submission will be graded primarily based on the following:

1. You will be graded on the basis of the quality of your responses to the questions.
2. Output screenshots.

Staff Graded Assignment -2
Graded Assignment • 5h

Due Nov 15, 11:59 PM IST

Instructions

1. Submit your Protobuf definition file (banking.proto) and a video recording of code explanation with your own voice over. — 20 points

   Upload File

   Add File

2. Submit your server-side implementation code and a video recording of code explanation. — 30 points

   Upload File

   Add File

3. Submit the client-side implementation code and a video recording of code explanation with your own voice over. — 30 points

   Upload File

   Add File

4. Submit the video recording of demonstration of your application with your own voice-over. — 20 points

   Upload File

   Add File